

PROBLEM SOLVING AND TESTING USING JAVA

WEEK 2

Running Sum of 1D Array

1480. Running Sum of 1d Array

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0..i])`. Return the running sum of `nums`.

Example 1:
Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:
Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:
Input: `nums = [3,1,2,10,1]`
Output: `[3,4,6,16,17]`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-10^6 \leq \text{nums}[i] \leq 10^6$

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] = nums[i] + nums[i - 1];
        }
        return nums;
    }
}
```

Testcase: **Accepted** Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `nums = [1,2,3,4]`

Output: `[1,3,6,10]`

Expected: `[1,3,6,10]`

Shuffle The Array

1470. Shuffle the Array

Given the array `nums` consisting of $2n$ elements in the form `[x1, x2, ..., xn, y1, y2, ..., yn]`. Return the array in the form `[x1, y1, x2, y2, ..., xn, yn]`.

Example 1:
Input: `nums = [2,5,1,3,4,7], n = 3`
Output: `[2,3,5,4,1,7]`
Explanation: Since $x_1=2, x_2=5, x_3=1, y_1=3, y_2=4, y_3=7$ then the answer is `[2,3,5,4,1,7]`.

Example 2:
Input: `nums = [1,2,3,4,3,2,1], n = 4`
Output: `[1,4,2,3,3,2,4,1]`

Example 3:
Input: `nums = [1,1,2,2], n = 2`
Output: `[1,2,1,2]`

Constraints:

- $1 \leq n \leq 500$
- `nums.length == 2n`
- $1 \leq \text{nums}[i] \leq 10^3$

```
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] ans = new int[n * 2];
        int index = 0;
        for (int i = 0; i < n; i++) {
            ans[index++] = nums[i];
            ans[index++] = nums[i + n];
        }
        return ans;
    }
}
```

Testcase: **Accepted** Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `nums = [2,5,1,3,4,7]`

`n = 3`

Output: `[2,3,5,4,1,7]`

Expected: `[2,3,5,4,1,7]`

Remove Element

The screenshot shows a coding problem titled "27. Remove Element". The problem description states: "Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` *in-place*. The order of the elements may be changed. Then return the number of elements in `nums` which are not equal to `val`." It also provides a custom judge with test cases and an example.

Example 1:
Input: `nums = [3,2,2,3]`, `val = 3`
Output: `2`, `nums = [2,2,...]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 2. It does not matter what you leave beyond the returned `k` (hence they are underscores).

The code editor shows a Java solution:

```
class Solution {
    public int removeElement(int[] nums, int val) {
        int k = 0;
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[k] = nums[i];
                k++;
            }
        }
        return k;
    }
}
```

The test result shows "Accepted" with a runtime of 0 ms. The input is `nums = [3,2,2,3]` and `val = 3`. The output is `[2,2]` and the expected output is `[2,2]`.

Remove Duplicates From Sorted Array

The screenshot shows a coding problem titled "26. Remove Duplicates from Sorted Array". The problem description states: "Given an integer array `nums` sorted in *non-decreasing order*, remove the duplicates *in-place* such that each unique element appears only *once*. The *relative order* of the elements should be kept the same." It also provides a custom judge with test cases and an example.

Example 1:
Input: `nums = [1,1,2]`
Output: `2`, `nums = [1,2,...]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2 respectively. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:
Input: `nums = [0,0,1,1,1,2,2,3,3,4]`
Output: `5`, `nums = [0,1,2,3,4,...]`

The code editor shows a Java solution:

```
class Solution {
    public int removeDuplicates(int[] nums) {
        int k = 1;
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[k - 1]) {
                nums[k] = nums[i];
                k++;
            }
        }
        return k;
    }
}
```

The test result shows "Accepted" with a runtime of 0 ms. The input is `nums = [1,1,2]`. The output is `[1,2]` and the expected output is `[1,2]`.

Maximum Subarray

53. Maximum Subarray

Given an integer array `nums`, find the **subarray** with the largest sum, and return its sum.

Example 1:
Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:
Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:
Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

Discover more [Data Structure Courses](#)

```
class Solution {
    public int maxSubarray(int[] nums) {
        int maxsum = nums[0];
        int currentsum = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currentsum = Math.max(nums[i], currentsum + nums[i]);
            maxsum = Math.max(maxsum, currentsum);
        }
        return maxsum;
    }
}
```

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`

Output: 6

Expected: 6

Contribute a testcase

Find The Highest Altitude

1732. Find the Highest Altitude

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length `n`, where `gain[i]` is the **net gain in altitude** between points `i` and `i + 1` for all $(0 \leq i < n)$. Return the **highest altitude** of a point.

Example 1:
Input: `gain = [-5,1,5,0,-7]`
Output: 1
Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:
Input: `gain = [-4,-3,-2,-1,4,3,2]`
Output: 8
Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$

Discover more [Search Engines](#)

Seen this question in a real interview before? 1/6

```
class Solution {
    public int largestAltitude(int[] gain) {
        int altitude = 0;
        int maxAltitude = 0;
        for (int g : gain) {
            altitude += g;
            maxAltitude = Math.max(maxAltitude, altitude);
        }
        return maxAltitude;
    }
}
```

Accepted Runtime: 0 ms

Case 1 Case 2

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Expected: 1

Contribute a testcase

Group Anagrams

49. Group Anagrams

Medium

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strs.length} \leq 10^4$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

22.4K 425 472 Online

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String str : strs) {
            char[] chars = str.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(str);
        }
        return new ArrayList<>(map.values());
    }
}
```

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["eat","tea","ate"],["bat"],["tan","nat"]]`

Expected: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Top K Frequent Elements

347. Top K Frequent Elements

Medium

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in **any order**.

Example 1:

Input: `nums = [1,1,1,2,2,3], k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1], k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2], k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- k is in the range `[1, the number of unique elements in the array]`.
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size.

19.8K 499 555 Online

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        // Step 1: Count frequencies
        Map<Integer, Integer> freqMap = new HashMap<>();
        for (int num : nums) {
            freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
        }

        // Step 2: Create buckets
        List<Integer>[] bucket = new ArrayList[nums.length + 1];

        for (int key : freqMap.keySet()) {
            int freq = freqMap.get(key);
            if (bucket[freq] == null) {
                bucket[freq] = new ArrayList<>();
            }
            bucket[freq].add(key);
        }

        // Step 3: Collect top k elements
        int[] result = new int[k];
        int index = 0;

        for (int i = bucket.length - 1; i >= 0 && index < k; i--) {
            if (bucket[i] != null) {
                for (int num : bucket[i]) {
                    result[index++] = num;
                    if (index == k) {
                        break;
                    }
                }
            }
        }

        return result;
    }
}
```

Accepted Runtime: 0 ms

Java Dequeue

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int k = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        HashMap<Integer, Integer> map = new HashMap<>();
        int max = 0;
        for (int i = 0; i < n; i++) {
            if (map.containsKey(arr[i])) {
                map.put(arr[i], map.get(arr[i]) + 1);
            } else {
                map.put(arr[i], 1);
            }
            if (i >= k) {
                int x = arr[i - k];
                if (map.get(x) == 1) {
                    map.remove(x);
                } else {
                    map.put(x, map.get(x) - 1);
                }
            }
            if (i >= k - 1) {
                if (map.size() > max) {
                    max = map.size();
                }
            }
        }
        System.out.println(max);
        sc.close();
    }
}
```

```
C:\Windows\System32\cmd.e  X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

D:\>javac Solution.java

D:\>java Solution
6 3
5 3 5 2 3 2
3

D:\>
```

Java Hashset

```
import java.util.*;
public class Hashset {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        HashSet<String> set = new HashSet<>();
        for (int i = 0; i < n; i++) {
            String first = sc.next();
            String second = sc.next();
            String pair = first + " " + second;
            set.add(pair);
            System.out.println(set.size());
        }
        sc.close();
    }
}
```

```
C:\Windows\System32\cmd.e  X + v
Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

D:\>javac Hashset.java

D:\>java Hashset
5
john tom
1
john mary
2
john tom
2
mary anna
3
mary anna
3

D:\>|
```

